Critical Evaluation of “A Subset Feature Elimination Mechanism for Intrusion Detection System”

Final Project — Data Science in Cyber (Dr. Uri Itai)

Adam Karain (ID 318407889) — June 2026

Evaluated source: H. Nkiama, S. Z. Mohd Said and M. Saidu, “A Subset Feature Elimination Mechanism for Intrusion Detection System,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 7, no. 4, 2016. Reference implementation: the GitHub repository *CynthiaKoopman/Network-Intrusion-Detection*. Dataset: NSL-KDD (mirror: *defcom17/NSL_KDD*). All code, data files and the executed notebook accompanying this report are available in the submission repository.

1. Summary of the Source

The problem being addressed. A network Intrusion Detection System (IDS) must decide, for every connection it observes, whether the traffic is benign or malicious. Modern networks generate traffic at a volume that makes both the speed and the accuracy of this decision critical: a slow detector cannot keep up, and a detector that misses attacks creates false confidence. The article addresses a specific facet of this problem — dimensionality. The NSL-KDD benchmark describes every connection with 41 features, and the authors argue that many of them are redundant or irrelevant, slowing down the classifier and potentially hurting its detection rate.

Why the problem is important. Feature selection in an IDS is not a cosmetic optimization. Fewer features mean cheaper feature extraction at wire speed, faster model retraining as traffic evolves, and — if done correctly — models that are easier for security analysts to interpret. The question of *which* features matter is itself of operational value, since it tells analysts what to monitor.

The proposed solution. The authors propose a two-stage feature-selection mechanism, applied separately to each of the four NSL-KDD attack categories (DoS, Probe, R2L, U2R): first a univariate ANOVA F-test filter (scikit-learn’s *SelectPercentile*), then Recursive Feature Elimination (RFE) wrapped around a decision-tree classifier. The pipeline selects 11–15 features per category out of the 121 columns that remain after one-hot encoding. The headline claims are that the reduced models reach approximately 99.7–99.9% accuracy, precision, recall and F-measure, and that training time drops by roughly an order of magnitude (e.g., 15.5 s to 0.96 s for the DoS class).

The dataset used. NSL-KDD (Tavallaee et al., 2009), the cleaned successor of KDD’99: 125,973 labelled training connections (*KDDTrain+*) and 22,544 test connections (*KDDTest+*), each with 41 features, an attack-type label (39 types grouped into DoS, Probe, R2L, U2R and normal) and a difficulty score. By deliberate design of its authors, *KDDTest+* contains 17 attack types that never appear in training — 29.2% of all attack records — so that the benchmark measures generalization to novel attacks, not only recognition of known ones.

The model and methodology employed. A CART decision tree (scikit-learn *DecisionTreeClassifier*, information-gain criterion) trained per attack category on one-hot-encoded, standardized features; feature selection as described above; evaluation reported as accuracy, precision, recall, F-measure and confusion matrices, stated to use 10-fold cross-validation.

2. Critical Evaluation

The main claims. We extracted four falsifiable claims. C1: the feature-selection mechanism improves classification performance (the article's Tables IV–V show improvements such as R2L accuracy rising from 97.03% with 41 features to 99.88% with 13). C2: the per-class detectors achieve ≈99.7–99.9% accuracy, precision and recall. C3: feature selection drastically reduces training time (Table IX). C4 (implicit but essential): the resulting models are a sound basis for a practical IDS.

Are the claims supported by the presented evidence? Internally, yes — and we reproduce them: under the article's own protocol our 10-fold cross-validation accuracy is 99.97% (DoS), 99.88% (Probe), 99.92% (R2L) and 99.95% (U2R) — the last figure matching the article to the second decimal place. The numbers are real, not fabricated. The problem is what they measure. Cross-validation folds are drawn from the same distribution as the training data, so this protocol answers "can the model re-recognize the attack types it was trained on?". The article's conclusions, however, speak about detecting intrusions in general — a claim the presented evidence cannot support.

Is the evaluation methodology appropriate? No, in three respects. First, the official test set is never used for what it was built for: every reported metric is, as far as the text and the reference implementation allow us to determine, computed within a single distribution. NSL-KDD's creators provided *KDDTest+* precisely to expose detectors to unseen attack types; an evaluation that bypasses it removes the hardest — and most operationally relevant — part of the benchmark. Second, the article relies on accuracy as its headline metric despite severe class imbalance (52 U2R records in 125,973, i.e. 0.04%): a trivial all-negative classifier reaches 99.96% accuracy on the U2R training task, so accuracy differences in the third decimal place carry no information about detection ability. Third, the article's bookkeeping is inconsistent: it reports 126,620 training and 22,850 test records, while the official files contain 125,973 and 22,544; its Table III lists 3,191 R2L test instances, yet its own R2L confusion matrix sums to 2,885.

A revealing detail. The row totals of the article's confusion matrices (7,460 DoS; 2,421 Probe; 2,885 R2L; 67 U2R) match the official *KDDTest+* category sizes exactly, yet the accompanying metrics are at cross-validation level. The reference implementation makes the likely mechanism explicit: its evaluation cells apply *cross_val_score* to the **test set itself**, i.e., models are retrained inside test-set folds. Under that procedure the model always trains on the very attack types it is scored on — novel attacks cease to be novel, and 97–99% figures for R2L/U2R follow automatically. When the same reference notebook also performs the honest experiment (train on *KDDTrain+*, predict *KDDTest+*), it obtains R2L recall of 312/2,885 ≈ 10.8% and U2R recall of 7/67 ≈ 10.4% — results its own text then largely sets aside.

Possible weaknesses and limitations. Beyond the evaluation protocol: class imbalance is never discussed or mitigated (no class weights, no resampling, no cost-sensitive analysis); the train/test prevalence shift (R2L is 0.8% of training but 12.8% of the test set) goes unmentioned; no variance or significance is attached to the third-decimal accuracy improvements credited to feature selection; and the released materials are incomplete (Section 4 of this report).

Are the conclusions justified? The speed conclusion (C3) is justified and we confirm it. The performance conclusions (C1 as an absolute gain, C2, C4) are not: they rest on an evaluation that cannot, by construction, detect the failure mode that matters most for an IDS. Our findings contradict the author's central conclusions, and the reason is methodological, not a failure to

reproduce: we reproduce their numbers first, then show that the intended benchmark tells a different story — R2L recall of 9.6% against a claimed 97.4%.

3. Feature Engineering Analysis

Was feature engineering performed, and which features were used? Yes, on two levels. The dataset itself is already heavily engineered: NSL-KDD's 41 features include basic TCP/IP fields (duration, protocol, service, flag, byte counts), content features extracted from payloads (failed logins, root shells, file creations), and statistical window features computed over the preceding 2 seconds (*count*, *serror_rate*, …) or the last 100 connections per host (*dst_host_**). The article adds three transformations: one-hot encoding of the three nominal features (*protocol_type*, *service*, *flag*; 41 → 121 columns), standardization to zero mean and unit variance, and the two-stage feature selection (ANOVA filter + RFE) that is its main contribution.

Transformations in our reproduction, and why. We replicate the article's preprocessing and add one transformation it lacks: *log1p* compression of the four heavy-tailed counters (*src_bytes*, *dst_bytes*, *duration*, *count*). The rationale is documented in the notebook: these features span up to nine orders of magnitude (median *src_bytes* 44 bytes, maximum ≈1.4 GB) with skewness up to 290; the transform brings skewness below 1 for the byte counters, which stabilizes the linear baseline while leaving tree models untouched (the transform is monotonic). One-hot encoding is the right choice for the nominal features — label encoding would fabricate an order, and target encoding would inject label information into the features. Standardization is fitted on the training set only; the reference implementation instead fits a separate scaler on the test set, a small but real protocol flaw (test statistics leak into preprocessing, and per-split scaling can mask distribution shift).

Redundancy in the system. The Spearman correlation screen in the notebook finds near-duplicate groups: the SYN-error rate appears at three aggregation levels (*serror_rate*, *srv_serror_rate*, *dst_host_serror_rate*; pairwise $\rho = 0.92$–0.97), *same_srv_rate* and *diff_srv_rate* are near mirror images ($\rho = -0.92$), and *dst_host_srv_count* tracks *dst_host_same_srv_rate* ($\rho = 0.92$). One would spot such redundancy exactly this way — a $|\rho| > 0.9$ screen, variance-inflation factors, or clustering features on $1-|\rho|$ — and tackle it by keeping one representative per cluster, by regularization, or by letting a wrapper method such as RFE prune the copies. To be fair to the article, this redundancy means its premise (41 features contain substantial redundancy) is correct; our criticism concerns what was measured, not the premise.

Was the feature-selection process meaningful? Mathematically, RFE around a decision tree iteratively removes the features with the smallest impurity contribution, which is a coherent (greedy) way to find a small high-signal subset; on the DoS class our independent run re-selects 8 of the article's 12 features. From the cybersecurity standpoint the selected features are exactly the signatures an analyst would name: *flag_S0* (half-open connections — the *neptune* SYN flood), *service_ecr_i* (ICMP echo replies — *smurf*), *wrong_fragment* (*teardrop*), plus the SYN-error window rates. The selection is interpretable and domain-plausible — and therein lies the caveat: these are signatures of the *specific attacks in the training catalog*. A feature set optimized to separate *neptune* from normal traffic has no reason to flag an *apache2* request flood, which is precisely the failure our error analysis documents.

Features that could improve performance. Payload-derived statistics (byte entropy, header anomalies), per-source aggregates over horizons longer than 2 seconds (failed logins per hour, fan-out), and cross-connection sequence features. None exist in NSL-KDD — an argument for

validating on modern datasets (CICIDS2017/2018, UNSW-NB15).

4. Reproducibility Analysis

Can the code be executed successfully? The article releases no code; reproduction depends on its prose plus the community reference implementation. The reference notebook (*DecisionTree_IDS.ipynb*) runs after minor version updates (it predates current scikit-learn APIs). Our own notebook executes end-to-end with fixed seeds ("Run All", approximately two minutes) and reproduces every number quoted in this report.

Are all required files and dependencies available? Partially. The NSL-KDD files are freely available (we vendor the three required files in *data/*). The reference repository's README advertises two notebooks — decision tree and random forest — but only the decision-tree notebook exists in the repository, so the random-forest variant is not reproducible from the released materials. The article specifies neither library versions nor seeds.

Do hidden preprocessing steps exist? Yes, several decisions material to the results are unstated in the article and only discoverable from the reference code: how one-hot encoding handles the six service categories that occur only in training; the fact that scaling is fitted per split (on the test set separately — a leak); and, most consequentially, what the reported metrics are computed on. The confusion-matrix row totals (test-set sizes) combined with cross-validation-level scores are only consistent with the procedure visible in the reference notebook: *cross_val_score* applied to *KDDTest+* itself.

Overall reproducibility. Mixed. The pipeline is built from standard, well-documented components and we could reproduce the published numbers to the second decimal — in that narrow sense reproducibility is excellent. But the evaluation protocol, the single most important methodological fact, is recoverable only by forensic reading of a third-party notebook, and the article's own record counts disagree with the official files. Our submission addresses these gaps: pinned dependencies, vendored data, fixed seeds, one documented protocol per result.

5. Experimental Results

Experiments performed. (i) A binary detector (normal vs. attack) trained on all 121 encoded features with three models — logistic regression, the article's decision tree, and a random forest — each evaluated under two protocols: Protocol A, a stratified 80/20 split inside *KDDTrain+* (statistically equivalent to the article's cross-validation), and Protocol B, the official split (train on *KDDTrain+*, evaluate on *KDDTest+*). (ii) A faithful re-run of the article's per-class pipeline (RFE with a decision tree, 13 features) scored under both its own protocol and Protocol B. (iii) An error analysis of the strongest model. **Modifications introduced:** the *log1p* transform (Section 3); RFE fitted on a stratified 25% sample with step 10 for compute reasons; the feature count fixed at 13 for all classes (the article uses 11–15).

Evaluation metrics and their justification. Precision (share of alerts that are real attacks — its complement drives analyst alert fatigue); recall, the IDS detection rate (every false negative is an undetected intrusion — in security the costlier error); F1 and F2 (F2 weights recall twice as heavily, encoding the asymmetric cost of a breach versus minutes of triage); the Matthews correlation coefficient, the most reliable single number under class imbalance because it uses all four confusion-matrix cells and cannot be inflated by majority-class guessing; and ROC-AUC, which is threshold-free and separates ranking quality from threshold placement. Accuracy is reported only

because the article uses it as its headline figure. Regression metrics do not apply; specificity is captured by the false-positive analysis; F0.5 encodes the opposite cost structure to the IDS use case and is therefore omitted.

Model	Protocol	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Logistic Regression	A	0.9729	0.9767	0.9649	0.9707	0.9672	0.9456	0.9965
Decision Tree	A	0.9984	0.9979	0.9986	0.9983	0.9985	0.9967	0.9984
Random Forest	A	0.9990	0.9994	0.9986	0.9990	0.9987	0.9981	1.0000
Logistic Regression	B	0.7544	0.9130	0.6284	0.7444	0.6702	0.5572	0.7638
Decision Tree	B	0.8117	0.9666	0.6931	0.8074	0.7347	0.6665	0.8309
Random Forest	B	0.7766	0.9687	0.6278	0.7618	0.6753	0.6168	0.9590

Table 1. Binary detectors under Protocol A (random split within the training distribution) and Protocol B (official KDDTest+).

Protocol A reproduces the tutorial-level picture — every model close to perfect. Under Protocol B the same models lose 18–22 accuracy points; recall collapses to 0.63–0.69 while precision stays above 0.91, i.e., the models stay quiet and miss more than a third of the attacks. The next table shows the same phenomenon inside the article's own per-class design.

Class	Article: acc / prec / rec (%)	Our CV accuracy (%)	KDDTest+: acc / recall (%)
DoS	99.90 / 99.69 / 99.79	99.97	93.7 / 86.7
Probe	99.80 / 99.37 / 99.37	99.88	91.1 / 67.7
R2L	99.88 / 97.40 / 97.41	99.92	79.3 / 9.6
U2R	99.95 / 99.70 / 99.69	99.95	99.5 / 34.3

Table 2. The article's Table IV claims versus our reproduction (decision tree, RFE, 13 features per class).

Error analysis. For the random forest under Protocol B: per-category recall is 82.7% (DoS), 72.8% (Probe), 4.1% (R2L) and 11.9% (U2R); recall on attack types seen in training is 76.5% versus 29.6% on novel types. The most-missed attacks are *guess_passwd* (all 1,231 instances missed), *warezmaster* (835), *processtable* (679), *mscan* (654) and *snmpguess* (331). The pattern is systematic, not random: misses concentrate (a) where training data was scarce — R2L and U2R had 995 and 52 training records — and (b) on novel types, including novel *DoS* attacks (*apache2*, *mailbomb*, *processtable*) despite near-perfect detection of known DoS: the models learned the signatures of *neptune* and *smurf*, not the concept of a flood. The cybersecurity implication is uncomfortable: the detector is weakest exactly where an adversary would operate — credential attacks that look like legitimate logins (R2L sessions terminate with a normal *SF* flag 97% of the time) and techniques invented after training-data collection. **False-positive/false-negative trade-off:** at the default threshold the forest raises 260 false alarms per 9,711 benign connections (2.7%) while missing ≈4,800 of 12,833 attacks. Given the cost asymmetry (analyst minutes versus a breach), and a healthy ranking reserve (ROC-AUC 0.959 despite 0.63 recall), the operating point should be moved toward recall — the F2 metric and a threshold tuned against an explicit cost matrix formalize this. The decision tree has no such reserve (AUC 0.831): its limitation is the model, not the threshold.

6. Conclusions

Key findings. The article's numbers are reproducible to the second decimal under its own protocol, and wrong as a description of IDS performance: on the benchmark's official test set the same pipeline detects 9.6% of R2L and 34.3% of U2R attacks. The gap is driven, in order, by novel attack types (29.2% of test attacks), extreme class imbalance, and feature selection tuned to known-attack signatures. Claim verdicts: C1 partially confirmed (performance is preserved with 13 of 121 columns in-distribution; the claimed absolute gains are within evaluation noise and were never tested under shift); C2 rejected as stated; C3 confirmed; C4 rejected for R2L, U2R and novel attacks generally.

Lessons learned. The evaluation protocol matters more than the model: three different classifiers all looked perfect under one protocol and all collapsed under the other. Cross-validation answers "did I fit this distribution?", not "will this work on next year's attacks?". Accuracy under imbalance is close to meaningless — a U2R detector with 99.5% accuracy misses two of three attacks. And reproduction work pays: every individual number in the article checks out, while the headline conclusion does not.

Strengths of the proposed solution. A real and clearly stated problem; transparent, standard tooling; an interpretable feature-selection outcome that independent runs largely re-derive; an honest, confirmed speed result.

Weaknesses. The evaluation never uses the official test split as intended — the single flaw from which every overstated claim follows; imbalance unaddressed; accuracy-led reporting; inconsistent dataset bookkeeping; incomplete released materials.

Suggestions for future improvement. Evaluate on *KDDTest+*/*KDDTest-21* as the primary benchmark; handle imbalance (class weights, resampling, cost-sensitive learning); tune thresholds against an explicit FP/FN cost matrix; add an anomaly-detection branch (e.g., Isolation Forest trained on normal traffic) for never-seen attack families; calibrate scores; and validate on modern traffic datasets.

7. Executive Summary

This project critically evaluates Nkiama, Said and Saidu (2016), “A Subset Feature Elimination Mechanism for Intrusion Detection System” (IJACSA 7(4)), together with its community reference implementation, on the NSL-KDD network-intrusion benchmark. The article proposes per-attack-class feature selection — an ANOVA univariate filter followed by Recursive Feature Elimination around a decision tree — and reports approximately 99.7–99.9% accuracy, precision and recall with only 11–15 of 121 encoded features, plus an order-of-magnitude training speed-up.

We rebuilt the full pipeline in a single reproducible notebook (fixed seeds, vendored data, pinned dependencies) and scored it under two protocols: the article’s own (cross-validation within the training distribution) and the benchmark’s intended one (train on *KDDTrain+*, evaluate on the official *KDDTest+*, in which 29.2% of attack records belong to 17 attack types absent from training). Under the article’s protocol we reproduce its results essentially exactly — for the U2R class to the second decimal (99.95%). Under the intended protocol the same detectors collapse: recall falls to 86.7% (DoS), 67.7% (Probe), 9.6% (R2L, against a claimed 97.4%) and 34.3% (U2R). A binary detector ensemble shows the same pattern (e.g., random forest: 99.9% random-split accuracy versus 62.8% test recall). Error analysis shows the misses are systematic: recall on novel attack types is 29.6% versus 76.5% on known types, and the single most-missed attack is password guessing — all 1,231 instances — because R2L traffic is statistically almost indistinguishable from legitimate sessions in NSL-KDD’s features.

Diagnosis: the article’s evaluation — made explicit by its reference implementation, which applies cross-validation to the test set itself — retrains models inside the distribution they are scored on, so novel attacks are never actually novel at evaluation time. Its claims are reproducible but answer the wrong question. The speed claim is confirmed; the performance and practicality claims are rejected. We do not recommend this pipeline, as published, as the basis of an IDS; with honest evaluation, imbalance handling, cost-aware thresholds and an anomaly-detection branch for unseen attacks, its interpretable feature-selection core remains a useful component. The broader lesson of the project: in security machine learning, the evaluation protocol — not the classifier — is where the truth lives.

8. Summing It Up

The problem: deciding which of NSL-KDD’s 41 traffic features an intrusion-detection classifier actually needs. The source: Nkiama et al. (2016), who answer with a per-class ANOVA + RFE + decision-tree pipeline and report ≈99.9% across all metrics; we also audited its reference implementation on GitHub. The dataset: NSL-KDD, whose official test set was deliberately constructed to contain attack types unseen in training. The methodology of our reproduction study: rebuild the pipeline end-to-end, reproduce the article’s numbers under its own cross-validation protocol, then re-score the identical models on the official test set, with three binary baselines (logistic regression, decision tree, random forest) as controls and a systematic error analysis.

Main findings: the published numbers reproduce almost exactly under the published protocol — and the protocol is the problem. It evaluates within the distribution the models were fitted on (the reference code cross-validates on the test set itself), which structurally hides the benchmark’s novel attacks. Under the intended evaluation, R2L recall is 9.6% versus the claimed 97.4%, U2R recall is 34.3%, and every model’s recall on novel attack types is less than half its recall on known ones. The author’s claims are therefore **not supported** by an appropriate evaluation, with one

exception: the training speed-up, which we confirm. The most important insights: evaluation protocol dominates model choice; accuracy under imbalance misleads (99.5% accuracy alongside 34% recall); and feature selection, while interpretable and fast, encoded signatures of known attacks rather than transferable structure. Recommendation: do not use this pipeline as published for similar problems; reuse its feature-selection core only inside an honest train/test protocol with imbalance handling, cost-aware thresholds and an anomaly-detection component. Final conclusion: the article is a textbook case — reproducible numbers, wrong question — and NSL-KDD's neglected official test set is exactly the instrument that reveals the difference.